std.vector: Advanced and Expert Guide

KingCobra standard library. Written against the `kingcobra.zip` you uploaded (`std/vector/vector.c`, `docs/STD_VECTOR.md`). Every behavior claim and code sample below was run on an interpreter built from that tree.

0. What std.vector is (and is not)

`std.vector` is a **math** module: N-dimensional vector algebra, quaternions and 3x3/4x4 matrices. It is not a growable container like C++ `std::vector`. For dynamic arrays use the language's built-in arrays; `std.vector` operates *on* them.

```
import std.vector;     // functions live under the `vector.` namespace
```

This guide skips the function catalogue (see `docs/STD_VECTOR.md`) and concentrates on what advanced users need: the data model, cost model, numerical edge cases, conventions that cause bugs, and production patterns.

1. Data model: no new types

Thing	Representation	Notes
vector (any dim)	array of numbers `[x, y, z, ...]`	0 to 4096 components
quaternion	`[x, y, z, w]`	**w last**, Hamilton product
mat3	flat array, 9 numbers	column-major, `m[col*3 + row]`
mat4	flat array, 16 numbers	column-major, `m[col*4 + row]`; translation in `m[12..14]`
angles	radians everywhere	use `deg2rad` / `rad2deg`
results	always **float** arrays	ints are accepted as input

Consequences you should design around:

1. **Value semantics.** Every function reads its inputs into a C `double` buffer and returns a *fresh* array. Inputs are never mutated and never aliased by results. You can safely hold a vector in two places.
2. **Everything is validated per call.** Each argument is checked to be an array of numbers (`add: expected a number, got string`), and fixed-size functions check the exact length (`expected 4 components, got 3`). A quaternion passed where a vec3 is expected is a hard error, not a silent truncation.
3. **Plain arrays work with the rest of the language.** Indexing, `len()`, `for` loops and destructuring (`let [x, y, z] = v;`) all apply. You can mutate a *result* in place (`m[5] = -1`) because it is yours.
4. **No type-level distinction between a vec4, a quaternion and a 4-component color.** Naming discipline (`q_`, `v_`, `m_`) is on you.

Tip: annotate return types. An un-annotated `fn` that returns an array is inferred as `void` by the type checker and rejected (`return type mismatch: function declared void, returning array`). Write `-> any` (or a concrete type) on helpers that return vectors.

2. Cost model

Each call does three things: unbox the arguments into `double[]`, compute, and box the result into a new array. For small vectors the interpreter call overhead dominates; for large vectors **boxing the result** dominates. Measured on the build from your zip (single run in a sandbox, so treat as relative numbers, not absolutes):

Operation	Time
200,000 x `add` on 3D vectors	34 ms (about 0.17 µs per call)
20,000 x `add` on 1024D vectors	268 ms (about 13 µs per call)
20,000 x `dot` on 1024D vectors	43 ms (about 2 µs per call)
100,000 x `lerp` (one call)	22 ms
100,000 x `add(a, scale(sub(b, a), t))` (same math, three calls)	55 ms

Rules of thumb that follow:

- **Reductions are cheap, vector-valued ops are expensive.** `dot`, `length_sq`, `distance_sq`, `sum` return a scalar and skip the result array. A 1024D `dot` is about six times cheaper than a 1024D `add`.
- **Prefer the fused primitive.** `lerp`, `move_towards`, `clamp_length`, `project`/`reject`, `reflect`, `refract`, `rotate3`, `mat4_trs` and `mat3_normal` each replace a chain of 2 to 6 calls and allocations. Use them.
- **Compare squared quantities.** `length_sq(v) < r*r` and `distance_sq(a, b) < r*r` avoid a `sqrt` and are the right form for radius tests in hot loops.
- **Hoist matrices out of loops.** Build `proj` and `view` once per frame, `mat4_mul` them once, and reuse the product for every vertex. Rebuilding a perspective matrix per object is a classic waste.
- **Batch with matrices, not loops of vectors.** If you must transform many points, a single `mat4_transform_point` per point beats composing several scalar ops per point.
- **Stack usage.** The C implementation keeps `double[4096]` scratch buffers (32 KB each) on the C stack. A binary op uses two of them (64 KB); `barycentric` uses seven (about 224 KB). Irrelevant on desktop threads, but worth knowing if you run on small embedded or thread stacks.

3. Numerical behavior reference

The module is deliberately forgiving in some places and strict in others. Knowing which is which prevents most bugs.

Situation	Behavior
`normalize` of the zero vector	returns the zero vector (no error)
`angle` with a zero-length input	returns `0`
`project(a, onto)` / `reject(a, from)` with a zero `onto`	projection is the zero vector; rejection returns `a`
`reflect`, `refract`	normalise the normal (and, for `refract`, the incident vector) internally; total internal reflection returns the **zero vector**
`quat_from_axis_angle`, `mat4_rotate`, `rotate3` with a zero axis	identity / no rotation, no error
`quat_normalize` of a zero quaternion	identity `[0,0,0,1]`
`quat_inverse` of a zero quaternion	**error**
`div` by zero	IEEE: `inf`, `-inf`, `nan` (no error). Check with `is_finite`
`lerp`, `quat_slerp` with `t` outside [0,1]	**extrapolates**; `t` is never clamped
`quat_slerp`	takes the shortest arc (negates one input if `dot < 0`); near-parallel inputs fall back to a normalised lerp
`equals(a, b)` with different lengths	returns `false` (every other binary op raises a dimension-mismatch error)
`equals` default tolerance	`1e-9`, **absolute**, not relative
`clamp(v, lo, hi)`	`lo` and `hi` are *scalars* applied to every component; `lo > hi` is an error
ray tests	return distance `t >= 0` or `null`
`mat4_inverse`, `mat3_inverse`, `mat3_normal`	error if `abs(det) < 1e-14` (absolute threshold)
`mat4_det`	never errors; returns the computed determinant even for singular matrices
`mat4_transform_point`	divides by `w` only when `w` is neither 0 nor 1

Three of these deserve a closer look.

Absolute epsilons are not scale-aware. The singularity test in the inverse functions compares `det` to `1e-14`. A perfectly valid uniform scale of `1e-5` has `det = 1e-15` and is reported as singular (verified: `mat4_inverse(mat4_scale(1e-5, 1e-5, 1e-5))` raises `matrix is singular`). If your world units are tiny, see the analytic inverse in section 6.4.

Overflow is silent. `length` squares components. For magnitudes near `1e154` and above, the squared sum overflows to `inf`, and `normalize` then divides by infinity and returns zeros. Keep magnitudes in a sane range, or pre-scale by the largest component.

`ray_sphere` semantics. The direction need not be unit length, and `t` is measured in units of that direction vector (a direction of length 2 gives half the distance). If the origin is inside the sphere the near root is negative, so the function returns the **far** root. `ray_plane` returns `null` both for parallel rays (`abs(dot(n, d)) < 1e-12`) and for planes behind the origin; you cannot tell the two apart.

4. Error handling

The reference doc says a bad call stops the program with a positioned message. In practice these are ordinary runtime errors, and `try`/`catch` receives an object:

```
import std.vector;

fn main() {
    try {
        vector.add([1, 2], [1, 2, 3]);
    } catch (e) {
        print(e.message);   // vector.add: dimension mismatch (2 vs 3)
        print(e.kind, e.line, e.col);
    }
}
```

The error classes you will actually see: dimension mismatch, wrong component count for fixed-size functions, non-numeric element, vector longer than 4096, `clamp` with `lo > hi`, singular matrix (`mat4_inverse`, `mat3_inverse`, `mat3_normal`), `quat_inverse` of zero, `barycentric` on a degenerate triangle, `mat4_look_at` with `eye == center` or `up` parallel to the view direction, and invalid `mat4_perspective` / `mat4_ortho` parameters.

Strategy. Do not wrap every math call in `try`. Validate at system boundaries (loaded assets, user input, network data) with `is_finite`, length checks and your own degenerate-case guards, then let the inner loops run unguarded and fail loudly if an invariant breaks.

5. Vector algebra patterns

5.1 N-dimensional work: similarity

Every vector function is dimension-agnostic, so the same API serves embeddings or feature vectors, not just geometry.

```
fn cosine(a: any, b: any) -> float {
    let d = vector.length(a) * vector.length(b);
    if (d == 0) { return 0.0; }
    return vector.dot(a, b) / d;
}

print(cosine([1, 2, 3, 4], [4, 3, 2, 1]));   // 0.666667
```

For many comparisons against one query, `normalize` the stored vectors once; cosine then reduces to a single `dot` per pair.

5.2 Orthonormal bases (Gram-Schmidt)

`reject` is the projection-removal step, so a basis from two non-parallel vectors is three lines:

```
fn basis(a: any, b: any) -> any {
    let x = vector.normalize(a);
    let y = vector.normalize(vector.reject(b, x));
    let z = vector.cross(x, y);          // right-handed
    return [x, y, z];
}
```

If `b` is parallel to `a`, `reject` yields the zero vector and `y` becomes zero, so guard that case when the inputs are not under your control.

5.3 2D `cross` is a signed area

For two 2D vectors `cross` returns the scalar `a.x*b.y - a.y*b.x`. Its sign is a cheap, branch-friendly test for orientation: positive means `b` is counter-clockwise of `a`. Use it for winding order, left/right-of-line tests and convex-polygon checks. `perp(v)` rotates 90 degrees counter-clockwise, which gives you a 2D normal without trigonometry.

5.4 Triple product

`triple(a, b, c)` is `a . (b x c)`: the signed volume of the parallelepiped. A sign flip tells you handedness changed; a value near zero tells you the three vectors are coplanar.

5.5 Point in triangle with a degenerate guard

`barycentric` raises on a zero-area triangle, so guard first when the triangle comes from data:

```
fn in_triangle(p: any, a: any, b: any, c: any) -> bool {
    let area2 = vector.cross(vector.sub(b, a), vector.sub(c, a));   // 2D: scalar
    if (vector.abs([area2])[0] < 1e-12) { return false; }           // degenerate
    let w = vector.barycentric(p, a, b, c);                         // [u, v, w]
    return w[0] >= 0 && w[1] >= 0 && w[2] >= 0;
}
```

The same weights interpolate any per-vertex attribute: `u*A + v*B + w*C` via `scale` and `add`.

5.6 Steering and smoothing

- `move_towards(cur, target, max_delta)` gives constant-speed motion that lands exactly on the target without overshoot (it returns `target` once within `max_delta`).
- `clamp_length(v, max)` caps a speed or force while preserving direction.
- `lerp(cur, target, k)` per frame is exponential smoothing, but it is **frame-rate dependent**. For frame-rate independence derive the factor from the timestep: `k = 1 - e^(-rate * dt)`, then `cur = lerp(cur, target, k)`.
- `smoothstep(e0, e1, x)` is a scalar easing function (cubic Hermite); it clamps its input, and when `e0 == e1` it degenerates to a step.

5.7 Reflection and refraction

`refract(v, n, eta)` follows GLSL semantics with `eta` as the ratio of refractive indices (incident over transmitted). Total internal reflection returns the zero vector, so test for it explicitly if you need to fall back to `reflect`:

```
let t = vector.refract(dir, normal, 1.0 / 1.5);
if (vector.length_sq(t) == 0) { t = vector.reflect(dir, normal); }
```

6. Matrices: conventions that cause bugs

6.1 Column-major and multiplication order

`mat4_mul(a, b)` returns `a * b`, which **applies `b` first**. With column vectors, read the product right to left:

```
let T = vector.mat4_translate(10, 0, 0);
let R = vector.mat4_rotate_z(vector.deg2rad(90));

// rotate, then translate:
vector.mat4_transform_point(vector.mat4_mul(T, R), [1, 0, 0]);   // [10, 1, 0]
// translate, then rotate:
vector.mat4_transform_point(vector.mat4_mul(R, T), [1, 0, 0]);   // [0, 11, 0]
```

For a full model-view-projection chain: `mvp = proj * view * model`.

6.2 The camera pipeline

```
let model = vector.mat4_trs([0, 0, -5], vector.quat_from_euler(0, 0.5, 0), [1, 1, 1]);
let view  = vector.mat4_look_at([0, 0, 0], [0, 0, -1], [0, 1, 0]);
let proj  = vector.mat4_perspective(vector.deg2rad(70), 1.6, 0.1, 100);
let mvp   = vector.mat4_mul(proj, vector.mat4_mul(view, model));

let ndc  = vector.mat4_transform_point(mvp, [0, 0, 0]);     // includes the w-divide
let clip = vector.mat4_mul_vec4(mvp, [0, 0, 0, 1]);         // raw clip space, no divide
```

`mat4_trs(t, q, s)` is exactly `T * R * S` in one call (verified equal to the manual composition). The matrices are right-handed with the camera looking down -Z.

Clip-space convention. `mat4_perspective` produces **OpenGL clip space**: depth maps to [-1, 1]. The column-major *layout* matches Vulkan, but the clip volume does not. Depth is also strongly non-linear (a point at `z = -1` with near 0.1 and far 100 lands at NDC depth about 0.80), so do not interpolate linear quantities in NDC. For Vulkan/Metal/D3D (depth [0, 1], Y down) post-multiply a fix-up matrix:

```
let fix = vector.mat4_identity();
fix[5]  = -1;      // flip Y
fix[10] = 0.5;     // z' = 0.5*z + 0.5*w
fix[14] = 0.5;
let proj_vk = vector.mat4_mul(fix, proj);
```

This maps the near plane to depth 0 and the far plane to 1 (verified).

6.3 Normals need the normal matrix

Under non-uniform scale, transforming a normal by the model matrix tilts it. `mat3_normal(model)` computes `transpose(inverse(upper-left 3x3))`:

```
let M = vector.mat4_mul(vector.mat4_rotate_y(0.5), vector.mat4_scale(1, 4, 1));
let N = vector.mat3_normal(M);
vector.mat3_mul_vec3(N, [0, 1, 0]);     // [0, 0.25, 0]: renormalise before lighting
```

The result is not unit length; `normalize` it after transforming. Use `mat4_transform_dir` for directions that should follow the model (translation ignored), and `mat4_transform_point` for positions.

6.4 Inverting safely

Because the singularity threshold is absolute, there are two situations to handle.

Tiny or huge uniform scale. For matrices you built as translation, rotation and scale, skip the determinant entirely and invert analytically as `S^-1 * R^T * T^-1`:

```
fn trs_inverse(t: any, q: any, s: any) -> any {
    let si = vector.mat4_scale(1.0 / s[0], 1.0 / s[1], 1.0 / s[2]);
    let rt = vector.quat_to_mat4(vector.quat_conjugate(q));
    let ti = vector.mat4_translate(-t[0], -t[1], -t[2]);
    return vector.mat4_mul(si, vector.mat4_mul(rt, ti));
}
```

Verified: it matches `mat4_inverse` on an ordinary matrix to 1e-12, and it round-trips a matrix with scale `1e-6` where `mat4_inverse` raises. It is also cheaper than a general 4x4 cofactor inverse and more accurate.

Detecting near-degenerate matrices independent of scale. `mat4_det` never raises, so you can build a scale-free test. By Hadamard's inequality `abs(det3)` is at most the product of the three column lengths, so the ratio below lies in [0, 1] and drops toward 0 when columns become linearly dependent, regardless of overall scale:

```
fn conditioning(m: any) -> float {
    var prod = 1.0;
    for (c in 0..3) { prod = prod * vector.length([m[c*4], m[c*4+1], m[c*4+2]]); }
    if (prod == 0) { return 0.0; }
    return vector.abs([vector.mat3_det(vector.mat3_from_mat4(m)) / prod])[0];
}
```

Verified: a uniform `1e-6` scale scores 1, a flattened `scale(1, 1, 0)` scores 0. Note this detects collinear columns, not anisotropy: `scale(1, 1, 1e-6)` also scores 1 because its columns are still orthogonal.

6.5 Picking: screen to world ray

Unproject two points on the NDC cube (near and far) through the inverse of `proj * view`. `mat4_transform_point` does the perspective divide for you:

```
fn unproject(inv: any, nx: float, ny: float, nz: float) -> any {
    return vector.mat4_transform_point(inv, [nx, ny, nz]);
}

fn pick_ray(view: any, proj: any, mx: float, my: float, w: float, h: float) -> any {
    let inv = vector.mat4_inverse(vector.mat4_mul(proj, view));
    let nx = 2.0 * mx / w - 1.0;
    let ny = 1.0 - 2.0 * my / h;                // screen Y grows downward
    let a = unproject(inv, nx, ny, -1);         // near plane (OpenGL clip space)
    let b = unproject(inv, nx, ny,  1);         // far plane
    return [a, vector.normalize(vector.sub(b, a))];
}

let view = vector.mat4_look_at([0, 2, 8], [0, 0, 0], [0, 1, 0]);
let proj = vector.mat4_perspective(vector.deg2rad(60), 16.0 / 9.0, 0.1, 100);
let r = pick_ray(view, proj, 960, 540, 1920, 1080);
vector.ray_sphere(r[0], r[1], [0, 0, 0], 1);   // 7.146...
```

If you pass Vulkan-style `proj_vk`, use `nz = 0` for the near plane and `ny` without the flip. Because the ray direction is unit length, the returned `t` is a true world distance.

7. Quaternions

7.1 Composition order

`quat_mul(a, b)` applies `b` first, then `a`, the same convention as `mat4_mul`. Rotations do not commute (verified: `Y*Z` and `Z*Y` differ).

```
let qa = vector.quat_from_axis_angle([0, 0, 1], vector.deg2rad(90));
let qb = vector.quat_from_axis_angle([1, 0, 0], vector.deg2rad(90));
vector.quat_rotate(vector.quat_mul(qa, qb), [0, 1, 0]);   // +Y -> (X90) -> +Z -> (Z90) -> +Z
```

7.2 Integrating angular velocity

Build a small rotation for the timestep and multiply on the **left** for a world-space angular velocity (on the right for a body-space one), then renormalise:

```
fn integrate(q: any, w: any, dt: float) -> any {
    let ang = vector.length(w) * dt;
    if (ang == 0) { return q; }
    let dq = vector.quat_from_axis_angle(w, ang);
    return vector.quat_normalize(vector.quat_mul(dq, q));
}

var q = vector.quat_identity();
for (i in 0..100) { q = integrate(q, [0, 0, 1.5707963], 0.01); }   // 1 s at 90 deg/s
vector.quat_rotate(q, [1, 0, 0]);                                  // about [0, 1, 0]
```

7.3 Normalisation discipline

What normalises for you and what does not:

Function	Normalises its input?
`quat_rotate`, `quat_to_mat4`, `mat4_trs`, `quat_to_euler`, `quat_slerp`	yes
`quat_mul`, `quat_conjugate`, `quat_inverse`	**no**

So `quat_rotate` tolerates a slightly non-unit quaternion, but a long chain of `quat_mul` calls accumulates error. With doubles the drift is slow (200,000 compositions stayed at length 1 to six digits), but long-lived state such as an object's orientation should still be `quat_normalize`d periodically. `quat_inverse` is a true inverse even for non-unit input (`q * inverse(q)` gives `[0,0,0,1]`); for unit quaternions prefer `quat_conjugate`, which is cheaper.

7.4 Interpolation

`quat_slerp` takes the shortest arc automatically: interpolating toward `neg(q)` gives the same rotation as toward `q`. Because `t` is unclamped, `t = 2` extrapolates (verified: from identity toward a 90 degree Z rotation, `t = 2` gives 180 degrees). For animation, clamp `t` yourself.

7.5 Euler angles: use at the edges only

`quat_from_euler(x, y, z)` and `quat_to_euler` use **roll = X, pitch = Y, yaw = Z**, applied Z then Y then X. Pitch is clamped through `asin`, and near +/-90 degrees pitch roll and yaw become coupled (gimbal lock), so the decomposition is ill-conditioned there. Keep orientation as a quaternion internally and convert only for UI or serialization.

8. Testing your math

With `equals(a, b, eps)` you can write property tests that catch convention mistakes early:

```
let I = vector.mat4_identity();
let m = vector.mat4_trs([1, 2, 3], vector.quat_from_euler(0.3, 0.2, 0.1), [2, 2, 2]);

assert(vector.equals(vector.mat4_mul(m, vector.mat4_inverse(m)), I, 1e-9));
let q = vector.quat_from_axis_angle([0, 0, 1], 1.0);
assert(vector.equals(vector.quat_mul(q, vector.quat_inverse(q)), [0, 0, 0, 1], 1e-12));
```

(`assert` here stands for whatever assertion helper your test setup uses.) Useful properties: `inverse` round trips, `length(quat_rotate(q, v)) == length(v)`, `dot(normalize(a), normalize(b))` within [-1, 1], `transform_dir` of an orthonormal

rotation preserves length, and `mat4_look_at` sends the eye to the origin. Always compare floats with a tolerance sized to your data, never `==`.

9. Expert checklist

1. Remember the layout: column-major, translation at indices 12 to 14, quaternion `w` last.
2. `mat4_mul(a, b)` and `quat_mul(a, b)` both apply **`b` first**.
3. `mat4_perspective` targets OpenGL clip space; add a fix-up matrix for [0,1] depth or Y-down APIs.
4. Transform positions with `transform_point`, directions with `transform_dir`, normals with `mat3_normal`.
5. Use the fused primitives and squared-distance comparisons in hot paths; hoist matrix products out of loops.
6. Absolute epsilons (`1e-9` in `equals`, `1e-14` in inverse, `1e-12` in `ray_plane`) assume unit-scale data. Rescale or use the analytic inverse for extreme magnitudes.
7. `lerp`, `slerp` and `smoothstep` do not all behave alike at the ends: `lerp` and `slerp` extrapolate, `smoothstep` clamps.
8. Guard degenerate inputs (`barycentric`, `look_at`, zero quaternions) at the boundary rather than catching errors in inner loops.
9. Renormalise accumulated quaternions; avoid Euler angles as stored state.
10. Annotate `-> any` on helper functions that return vectors.

10. Where to go next

- `docs/STD_VECTOR.md`: the compact function reference.
- `examples/std_vector.kc`: a runnable tour of the common calls.
- `std/vector/vector.c`: about 800 lines of plain C, short enough to read end to end if you want to confirm any behavior in this guide.